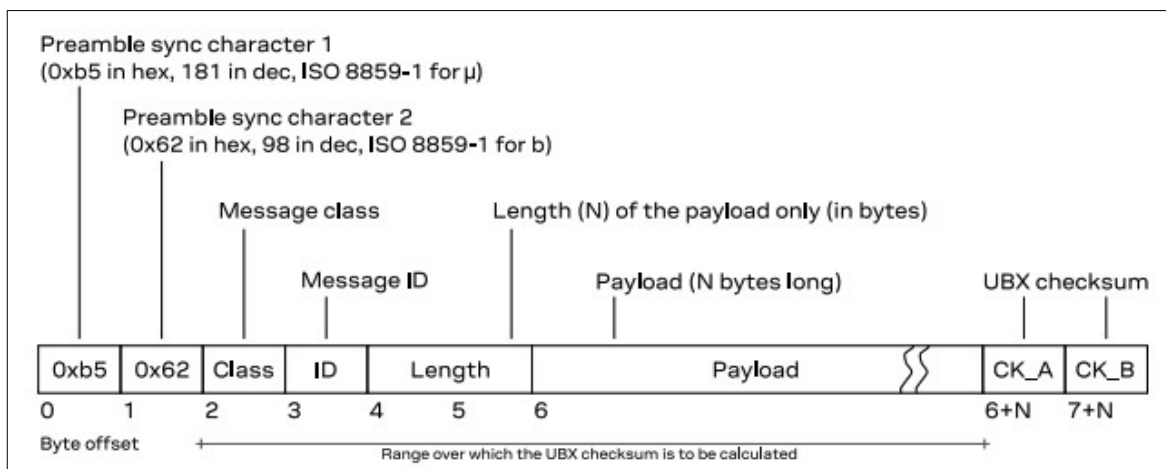


UBX format

UBX is a company specialising in GPS & RF technologies for embedded systems. This document concerns the use of UBX GPS binary protocol, A propriety Protocol used by all UBX GPS receivers such as the M9N.

The UBX protocol makes it more efficient for a MCU to have access to the GPS data compared to unpacking ASCII protocol bytes into numerical values, using the UBX protocol these can be unpacked at a lower CPU load with more deterministic behaviour. In this document the message from M9N NAV_IPT will be decoded since its the minimum and most important message of the module to be used in embedded devices like the pico for GPS application.

B5 62 – sync chars These two bytes exist for one reason, UART is a byte stream with no framing. If you plug in mid transmission you have no idea where the message ends and the next begins. The receiver scans the raw steam for B5 62 that pair only appears at the start of a message, so its a start-here maker.



To unpack these messages its important to know what amount of data is being received by the receiver, this is defined in the (**U-blox M9 SPG 4.04, Section 3.15.11**). The table below summarises the Data type & size.

Data	Size	Shift position
Sync	2	0 – 1
Class	1	2
ID	1	3
Length	2	4 – 5
Payload	90	
ACK	2	6

The data being unloaded is in order, since UART is a stream its important to feed it in a manner which doesn't allow the buffer to over flow or skip a byte and verifies along each step. Generally this in code is slightly complex since it usually consist of hard to understand state machines, In the case of this application the Check sum will be used to organise input bytes into a array. The array can then be parsed into a struct using pointers for easier access to the data from other functions or cores given that the application is programmed in C99.

The table below describes the offsets, size and description of the data in every byte in the Load array.

Fletchers running check sum

This algorithm is a extremely useful algorithm for asynchronous data transmission, for receiving UART messages properly verifying the message can be done in multiple ways, one is to buffer it into a string and parse it into values, this method is the most intuitive and logical but at the same time more primitive and any verification is more time costly since the verification is done after the sorting. Generally UART protocol is known to have dropped bits on the RX side of the transmission so to avoid any corrupted data along the transmission line a Fletchers Running check sum can be used.

$$ca = b_1 + b_2 + b_3 + \dots + b_n \quad (\text{every byte counted once})$$
$$cb = n \cdot b_1 + (n-1) \cdot b_2 + (n-2) \cdot b_3 + \dots + 1 \cdot b_n \quad (\text{position-weighted!})$$

in program:

```
(1) ca = b_0 → cb = ca | ca = b_0 , cb = b_0
(2) ca += b_1 → cb += ca | ca = b_0 + b_1 , cb = b_0 + b_0 + b_1
(3) ca += b_2 → cb += ca | ca = b_0 + b_1 , cb = b_0 + b_0 + b_0 + b_1 + b_1 + b_2
```

The process above describes how the values increment over cycles, it can be seen that byte_0 for cb multiplies by the amount of cycles and N bytes becomes the number of cycle minus their shift position. This gives a consistency between 2 values which contain variables.

For NAV_PVT messages on the M9N module the sender does this exact operation:



The initial check sum starts at a point where theres variables present, in this case it starts after the second sync byte (class) and ends at the end of the payload, after payload is received the check sum in the receiver side is completed, the last 2 bytes from the sender are compared to the checksums of the receiver, if both are the same the transmission was a success so now theres a complete string which holds all the variables needed.

Code application:

```
Switch (st) {
case dat1:
    if (byte == dat1){
        st = S_SYNC2;
        ca = byte; cb = ca;
    }
case dat2:
    if (byte == dat2){
        st = dat3;
    } else st = dat_1;
    ca += byte; cb += ca;
case dat3:
```

This is the main mechanism if applied in C, Generally theres many ways to do this checksum in terms of Syntax. Using cases is the more approachable manner since it takes each case at a time instead of looping over and checking each case. The code used above is just a example of how checksum is applied not the actual checksum used for the M9N. Keep this as a theoretical background since the actual application is very messy and not easy to understand unless the theory is understood first.

Parsing the Payload

Parsing the payload is a very straight forward approach but needs to be well defined so no buffers shift into other register values. The first is to define the sizes and values of the variables then their shift position. Using the table() in the next page each shift offset can be found, size of the value, scale and unit. The parsing is done in the Typical ansi C approach, by shifting bits and inserting bytes from a global array into a struct using pointers.

Example:

```
Static uint8_t frame[7] = {4_BYTE1, 4_BYTE2 ,4_BYTE3, 4_BYTE4, 2_BYTE1, 2_BYTE2, 1_BYTE};  
Static void decode(uint8_t *f, struct *s) {  
    s->4_byte =(uint32_t)f[0] | ((uint32_t)f[1]<<8) | ((uint32_t)f[2]<<16) | (((uint32_t)f[3]<< 24);  
    s->2_byte =(uint16_t)f[4] | ((uint16_t)f[5] << 8);  
    s->1_byte = f[6];  
}
```

The sketch splits a string of 7 bytes in a array and parses it into their designated memory type and stores them in a struct which can be used by any function as a variable.

The working mechanism is shifting each byte in a 32bit memory register then using a OR (|) operand to reconstruct them into a struct register. In a more explicit manner:

4_byte example:

```
(uint32_t)f[0]= 00000000 00000000 00000000 00001001 //  
(uint32_t)f[1]= 00000000 00000000 00000000 00010001 // 0x11  
(uint32_t)f[2]= 00000000 00000000 00000000 00100001 // 0x21  
(uint32_t)f[3]= 00000000 00000000 00000000 01000001 // 0x41  
  
(uint32_t)f[0] << 0  = 00000000 00000000 00000000 00001001  
(uint32_t)f[1] << 8  = 00000000 00000000 00010001 00000000  
(uint32_t)f[2] << 16 = 00000000 00100001 00000000 00000000  
(uint32_t)f[3] << 24 = 01000001 00000000 00000000 00000000  
_____  
s->4_byte          = 01000001 00100001 00010001 00001001 ← 32bit value from 4 8 bits
```

bitwise OR (|)

This is the basis on how to parse the full payload string into the struct, the information it holds can be scaled in a function after when used where the struct holds the raw values emitted from the transmitter. The Main things to look for are:

The table below shows the size and the offset of each set of memory in the message, this is used for unloading. Within this data theres flags, fixes and other information other than latitude & longitude, this is especially useful in the program since it gives information on the quality of the GPS receiver or serial transmitter in the case of the program. Next chapter will cover the usage of the information being sent.

Byte offset	Type	Name	Scale	Unit	Description
0	U4	iTOW	–	ms	GPS time of week of the navigation epoch (see iTOW timestamps section)
4	U2	year	–	y	Year (UTC)
6	U1	month	–	month	Month, range 1..12 (UTC)
7	U1	day	–	d	Day of month, range 1..31 (UTC)
8	U1	hour	–	h	Hour of day, range 0..23 (UTC)
9	U1	min	–	min	Minute of hour, range 0..59 (UTC)
10	U1	sec	–	s	Seconds of minute, range 0..60 (UTC)
11	X1	valid	–	–	Validity flags
11	U1	validDate (bit 0)	–	–	1 = valid UTC Date
11	U1	validTime (bit 1)	–	–	1 = valid UTC time of day
11	U1	fullyResolved (bit 2)	–	–	1 = UTC time of day has been fully resolved (no seconds uncertainty). Cannot be used to check if time is completely solved.
11	U1	validMag (bit 3)	–	–	1 = valid magnetic declination
12	U4	tAcc	–	ns	Time accuracy estimate (UTC)
16	I4	nano	–	ns	Fraction of second, range -1e9..1e9 (UTC)
20	U1	fixType	–	–	GNSS fix type: 0 = no fix, 1 = dead reckoning only, 2 = 2D-fix, 3 = 3D-fix, 4 = GNSS + dead reckoning combined, 5 = time only fix
21	X1	flags	–	–	Fix status flags
21	U1	gnssFixOK (bit 0)	–	–	1 = valid fix (i.e. within DOP & accuracy masks)
21	U1	diffSoln (bit 1)	–	–	1 = differential corrections were applied
21	U1	psmState (bits 4..2)	–	–	Power save mode state: 0 = PSM is not active, 1 = Enabled (intermediate state before Acquisition), 2 = Acquisition, 3 = Tracking, 4 = Power Optimized Tracking, 5 = Inactive
21	U1	headVehValid (bit 5)	–	–	1 = heading of vehicle is valid, only set if the receiver is in sensor fusion mode
21	U1	carrSoln (bits 7..6)	–	–	Carrier phase range solution status: 0 = no solution, 1 = floating ambiguities, 2 = fixed ambiguities (not supported for protocol versions less than 20.00)
22	X1	flags2	–	–	Additional flags
22	U1	confirmedAvai (bit 5)	–	–	1 = information about UTC Date and Time of Day validity confirmation is available (supported in protocol versions 19.00, 19.10, 20.10, 20.20, 20.30, 22.00, 23.00, 23.01, 27 and 28)
22	U1	confirmedDate (bit 6)	–	–	1 = UTC Date validity could be confirmed
22	U1	confirmedTime (bit 7)	–	–	1 = UTC Time of Day could be confirmed
23	U1	numSV	–	–	Number of satellites used in Nav Solution
24	I4	lon	1e-7	deg	Longitude
28	I4	lat	1e-7	deg	Latitude
32	I4	height	–	mm	Height above ellipsoid
36	I4	hMSL	–	mm	Height above mean sea level
40	U4	hAcc	–	mm	Horizontal accuracy estimate
44	U4	vAcc	–	mm	Vertical accuracy estimate
48	I4	veLN	–	mm/s	NED north velocity
52	I4	veLE	–	mm/s	NED east velocity
56	I4	veLD	–	mm/s	NED down velocity
60	I4	gSpeed	–	mm/s	Ground speed (2-D)
64	I4	headMot	1e-5	deg	Heading of motion (2-D)
68	U4	sAcc	–	mm/s	Speed accuracy estimate
72	U4	headAcc	1e-5	deg	Heading accuracy estimate (both motion and vehicle)
76	U2	pDOP	0.01	–	Position DOP
78	X2	flags3	–	–	Additional flags
78	U1	invalidLLh (bit 0)	–	–	1 = Invalid lon, lat, height and hMSL
78	U1	lastCorrectionAge (bits 4..1)	–	–	Age of most recently received differential correction: 0 = not available, 1 = 0–1 s, 2 = 1–2 s, 3 = 2–5 s, 4 = 5–10 s, 5 = 10–15 s, 6 = 15–20 s, 7 = 20–30 s, 8 = 30–45 s, 9 = 45–60 s, 10 = 60–90 s, 11 = 90–120 s, ≥12 = ≥120 s
78	U1	authTime (bit 13)	–	–	Time authentication flag: 0 = time is not authenticated, 1 = time is authenticated
80	U1\ [4]	reserved0	–	–	Reserved
84	I4	headVeh	1e-5	deg	Heading of vehicle (2-D); only valid when headVehValid is set, otherwise output is set to heading of motion
88	I2	magDec	1e-2	deg	Magnetic declination. Only supported in ADR 4.10 and later.
90	U2	magAcc	1e-2	deg	Magnetic declination accuracy. Only supported in ADR 4.10 and later.

